Personal ACM / ICPC Template

个人竞赛模板 • 从历史代码提取，面向铜牌巩固与银牌备赛

42 个模板 / 7 个章节 / C++17 / int + long long

查找：下一页目录列出每个模板的起始页；PDF 书签可直接跳转。

复制：KMP、Manacher 保留原完整程序；其他模板按各文件说明调用。

组合：N、g、init、build、query 等会重名，只复制需要的部分或手动改名。

下标：数组/图通常 1-index；KMP 输出 1-index，离散化/LIS 为 0-index。

容量：N 包含余量，最大合法数组下标是 N-1；先按题目改 N/LOG。

多测：调用 init/build；图的 g[1..旧n]、反图和边表需要自己清空。

数值：总和与乘积需要在 ll 范围内；INF 不是真正无限大。

写法：已有算法保留原实现；乘法用 ll，使用前核对模板注明的范围。

注释：新增算法附调用例子、变量含义和关键步骤；例子写在代码旁。

入门优先：树状数组、线段树、KMP、Manacher、最短路、基础数论。

进阶选读：匹配、LCA、SCC、割点/桥；A* 是第 k 短游走。

基础速记：sum(l,r)=prefix[r]-prefix[l-1]；prefix[0]=0。

一维差分：d[l]+=v，d[r+1]-=v；对 d 求前缀和还原。

gcd：std::gcd(a,b)；lcm 先除再乘，注意乘积范围。

维护：编辑分类目录中的 cpp；同步 catalog.json，验证后重新生成 PDF。

本册保留原变量与函数习惯；具体来源和修正记录见 template/README.md。

目录

1 Base / 基础

1.1 比赛主程序 3
1.2 快速幂 3
1.3 整数二分 3
1.4 二维前缀和 4
1.5 二维差分 4
1.6 离散化 4

2 Data Structure / 数据结构

2.1 并查集 5
2.2 树状数组 5
2.3 线段树 6
2.4 懒标记线段树 7
2.5 赋值加法线段树 8
2.6 ST 表 10
2.7 单调栈 10
2.8 单调队列 11

3 Graph / 图论

3.1 BFS 11
3.2 迭代 DFS 12
3.3 拓扑排序 13
3.4 Dijkstra 14
3.5 Floyd 15
3.6 Kruskal 16
3.7 二分图染色 17
3.8 二分图最大匹配 18
3.9 倍增 LCA 19
3.10 强连通分量 20
3.11 割点与桥 21
3.12 A* 第 k 短路 22

4 String / 字符串

4.1 KMP 23
4.2 Manacher 24
4.3 Trie 25
4.4 Z-function 26
4.5 双模字符串哈希 27

5 Math / 数学

5.1 埃氏筛 27
5.2 试除分解质因数 28
5.3 扩展欧几里得与逆元 28
5.4 组合数 29

6 DP / 动态规划

6.1 01 背包 29
6.2 完全背包计数 30
6.3 多重背包 30
6.4 分组背包 31
6.5 最长严格上升子序列 31
6.6 子集枚举 32

7 Geometry / 几何

7.1 二维凸包 33

1.1 比赛主程序

多测入口 | 按题目 | 历史提取/修正
按题意开启多测；每次 solve 清空使用过的全局状态

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
void solve()
{
}
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    int T = 1; //cin>>T;
    while(T--) solve();
    return 0;
}
```

1.2 快速幂

正模数下幂取模 | O(log n) | 历史提取/修正
0<=a<p，b>=0，1<=p<=3037000500；保留 ll 乘法，结果再 %p

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
ll a,b,p;
ll qpow(ll a , ll n)
{
    ll res=1;
    while(n>0){
        if(n%2==1) res=(res*a)%p;
        a=(a*a)%p;
        n/=2;
    }
    return res;
}
// 先设置全局 p，调用 qpow(a,b)%p。
```

1.3 整数二分

寻找最后一个不满足 check 的整数 | O(log 区间长度) 次 check | 历史提取/修正
0<=l<r，l+r 不溢出；check 先假后真，端点为假/真哨兵

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// 在题目代码中定义 check；真表示答案应向左找。
bool check(ll mid);
ll binary_search(ll l, ll r){
    while(l+1!=r){
        ll mid = (l+r)>>1;
        if(check(mid)) r = mid;
        else l = mid;
    }
    return l;
}
```

1.4 二维前缀和

静态子矩形求和 | 预处理 O(nm)，查询 O(1) | 历史提取/修正
先把原数组放 diff[1..n][1..m]，build 原地求前缀和；不能重复 build

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 1005;
ll diff[N][N];
void build(int n, int m){
    for(int i=0 ; i<=n ; i++) diff[i][0] = 0;
    for(int j=0 ; j<=m ; j++) diff[0][j] = 0;
    for(int i=1 ; i<=n ; i++){
        for(int j=1 ; j<=m ; j++){
            diff[i][j] -= diff[i-1][j-1];
            diff[i][j] += diff[i-1][j];
            diff[i][j] += diff[i][j-1];
        }
    }
}
ll query(int x1, int y1, int x2, int y2){
    return diff[x2][y2]-diff[x2][y1-1]
          -diff[x1-1][y2]+diff[x1-1][y1-1];
}
```

1.5 二维差分

离线矩形加，最终还原 | 修改 O(1)，还原 O(nm) | 历史提取/修正
1-index，n+1,m+1<N；原数组用 update(i,j,i,j,aij) 加入

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 1005;
ll d[N][N],a[N][N];
void init(){ memset(d,0,sizeof(d)); memset(a,0,sizeof(a)); }
void update(int x1, int y1, int x2, int y2, ll v){
    d[x1][y1] += v; d[x2+1][y1] -= v;
    d[x1][y2+1] -= v; d[x2+1][y2+1] += v;
}
void build(int n, int m){
    for(int i=1 ; i<=n ; i++){
        for(int j=1 ; j<=m ; j++){
            a[i][j] = a[i-1][j]+a[i][j-1]-a[i-1][j-1]+d[i][j];
        }
    }
}
```

1.6 离散化

值映射为有序编号 | 预处理 O(n log n)，查询 O(log n) | 历史提取/修正
先 x.clear() 并加入所有值；编号从 0 开始，不保留原坐标距离

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
vector<ll> x;
void init(){
    sort(x.begin(),x.end());
    x.erase(unique(x.begin(),x.end()),x.end());
}
int getidx(ll q){
    return lower_bound(x.begin(),x.end(),q)-x.begin();
}
```

2.1 并查集

动态连通性 | 单次 root/merge 最坏 O(n)，仅路径压缩 | 历史提取/修正
0<=点编号<n<N；保留原直接合并，长父链可能递归较深

```
// 不做按大小合并；单次 root 最坏 O(n)，递归长链要注意栈深
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 200005;
int fa[N];
void init(int n){
    for(int i=0 ; i<n ; i++) fa[i] = i;
}
int root(int x){
    if(fa[x] == x) return x;
    return fa[x] = root(fa[x]);
}
void merge(int x, int y){
    if(root(x) != root(y)) fa[root(x)] = root(y);
}
```

2.2 树状数组

单点加、前缀和、区间和 | 修改/查询 O(log n) | 历史提取/修正
n<N；update 的 x 必须>=1；普通版和差分版不能混用

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 5e5+9;
ll a[N],t[N];
int n,m;
int lowbit(int x){
    return x&-x;
}
// 多测时先清空树状数组，再按所需语义加入 a。
void init(int m){ n = m; fill(t,t+n+1,0); }
void update(int x, ll v){
    for(int i=x ; i<=n ; i+=lowbit(i)) t[i]+=v;
}
ll getsum(int x){
    ll res=0;
    for(int i=x ; i>0 ; i-=lowbit(i)) res+=t[i];
    return res;
}
// 原题用的是差分树状数组（区间加、单点查）：
// for(int i=1 ; i<=n ; i++) update(i,a[i]),update(i+1,-a[i]);
// 区间 [l,r] 加 v：update(l,v),update(r+1,-v)；单点 x：getsum(x)。
// 普通树状数组则只用 update(i,a[i]) 建立：单点加 update(x,v)，
// 前缀和 getsum(x)，区间和如下。普通版和差分版不能混用。
ll query(int l, int r){ return getsum(r)-getsum(l-1); }
```

2.3 线段树

单点赋值、区间和 | 建树 O(n)，操作 O(log n) | 历史提取/修正
a[1..n]，1<=n<N；build(1,1,n)，update1/query 从根调用

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 100005;
ll a[N],w[4*N];
void pushup(const int u){
    w[u] = w[2*u]+w[2*u+1];
}
void build(const int u, int L, int R){
    if(L==R){
        w[u] = a[L];
        return;
    }
    int M = (L+R)>>1;
    build(2*u,L,M);
    build(2*u+1,M+1,R);
    pushup(u);
}
ll query1(int u, int L, int R, int p){
    if(L==R){
        return w[u];
    }else{
        int M = (L+R)/2;
        if(M>=p) return query1(2*u,L,M,p);
        else return query1(2*u+1,M+1,R,p);
    }
}
void update1(int u, int L, int R, int p, ll x){
    if(L==R) w[u] = x;
    else{
        int M = (L+R)/2;
        if(M>=p)
            update1(2*u,L,M,p,x);
        else
            update1(2*u+1,M+1,R,p,x);
        pushup(u);
    }
}
bool InRange(int L, int R, int l, int r){
    return (l<=L && R<=r);
}
bool Outrange(int L, int R, int l, int r){
    return (L>r || R<l);
}
// 沿用原 query 的三种区间关系；单点修改版不需要懒标记下传。
ll query(int u, int L, int R, int l ,int r){
    if(InRange(L,R,l,r)) return w[u];
    else if(!Outrange(L,R,l,r)){
        int M = (L+R)/2;
        return query(2*u,L,M,l,r)+query(2*u+1,M+1,R,l,r);
    }else return 0;
}
```

2.4 懒标记线段树

区间加、区间和 | 建树 O(n)，操作 O(log n) | 历史提取/修正
a[1..n]，1<=n<N；build(1,1,n) 会重置标记；区间合法且非空

```
// 保留原 InRange/Outrange、maketag、pushdown、update/query：区间和需在 ll 内
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 100005;
ll a[N],w[4*N],lzy[4*N];
void pushup(const int u){
    w[u] = w[2*u]+w[2*u+1];
}
void build(const int u, int L, int R){
    lzy[u] = 0; // 多测重建时清掉上一次的标记。
    if(L==R){
        w[u] = a[L];
        return;
    }
    int M = (L+R)>>1;
    build(2*u,L,M);
    build(2*u+1,M+1,R);
    pushup(u);
}
bool InRange(int L, int R, int l, int r){
    return (l<=L && R<=r);
}
bool Outrange(int L, int R, int l, int r){
    return (L>r || R<l);
}
void maketag(int u, int len, ll x){
    lzy[u] += x;
    w[u] += len*x;
}
void pushdown(int u, int L, int R){
    int M = (L+R)/2;
    maketag(2*u,M-L+1,lzy[u]);
    maketag(2*u+1,R-M,lzy[u]);
    lzy[u] = 0;
}
ll query(int u, int L, int R, int l ,int r){
    if(InRange(L,R,l,r)) return w[u];
    else if(!Outrange(L,R,l,r)){
        pushdown(u,L,R);
        int M = (L+R)/2;
        return query(2*u,L,M,l,r)+query(2*u+1,M+1,R,l,r);
    }else return 0;
}
void update(int u, int L, int R, int l, int r, ll x){
    if(InRange(L,R,l,r)){
        maketag(u,R-L+1,x);
    }else if(!Outrange(L,R,l,r)){
        int M = (L+R)/2;
        pushdown(u,L,R);
        update(2*u,L,M,l,r,x);
        update(2*u+1,M+1,R,l,r,x);
        pushup(u);
    }
}
```

2.5 赋值加法线段树

区间赋值、区间加、区间最大值 | 建树 O(n)，操作 O(log n) | 历史提取/修正
1<=n<N；先下传赋值再下传加法；值域严格位于 (-INF,INF)

```
// 保留原 changed 标记和 change/add 两套下传函数；累积加法标记也要在 ll 内
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 1000005;
const ll INF = 0x3f3f3f3f3f3f3f3f;
ll a[N],w[4*N];
ll add_lzy[4*N];
ll change_lzy[4*N];
bool changed[4*N];
void pushup(const int u){
    w[u] = max(w[2*u],w[2*u+1]);
}
void build(const int u, int L, int R){
    add_lzy[u] = change_lzy[u] = 0; // 多测重建时清掉上一次的标记。
    changed[u] = false;
    if(L==R){
        w[u] = a[L];
        return;
    }
    int M = (L+R)/2;
    build(2*u,L,M);
    build(2*u+1,M+1,R);
    pushup(u);
}
bool InRange(int L, int R, int l, int r){
    return (l<=L) && (R<=r);
}
bool OutRange(int L, int R, int l, int r){
    return (R<l) || (L>r);
}
void add_maketag(int u, int len, ll x){
    add_lzy[u] += x;
    w[u] += x;
}
void change_maketag(int u, int len, ll x){
    add_lzy[u] = 0;
    change_lzy[u] = x;
    changed[u] = true;
    w[u] = x;
}
void add_pushdown(int u, int L, int R){
    int M = (L+R)/2;
    add_maketag(2*u,M-L+1,add_lzy[u]);
    add_maketag(2*u+1,R-M,add_lzy[u]);
    add_lzy[u] = 0;
}
void change_pushdown(int u, int L, int R){
    int M = (L+R)/2;
    if(changed[u]){
        change_maketag(2*u,M-L+1,change_lzy[u]);
        change_maketag(2*u+1,R-M,change_lzy[u]);
        change_lzy[u] = 0;
        changed[u] = false;
    }
}
```

2.5 赋值加法线段树（续）

01_data_structure/lazy_assign_add_max.cpp

```
void add(int u, int L, int R, int l, int r, ll x){
    if(InRange(L,R,l,r)){
        add_maketag(u,R-L+1,x);
    }else if(!OutRange(L,R,l,r)){
        int M = (L+R)/2;
        change_pushdown(u,L,R);
        add_pushdown(u,L,R);
        add(2*u,L,M,l,r,x);
        add(2*u+1,M+1,R,l,r,x);
        pushup(u);
    }
}
void change(int u, int L, int R, int l, int r, ll x){
    if(InRange(L,R,l,r)){
        change_maketag(u,R-L+1,x);
    }else if(!OutRange(L,R,l,r)){
        int M = (L+R)/2;
        change_pushdown(u,L,R);
        add_pushdown(u,L,R);
        change(2*u,L,M,l,r,x);
        change(2*u+1,M+1,R,l,r,x);
        pushup(u);
    }
}
ll query(int u, int L, int R, int l, int r){
    if(InRange(L,R,l,r)){
        return w[u];
    }else if(!OutRange(L,R,l,r)){
        int M = (L+R)/2;
        change_pushdown(u,L,R);
        add_pushdown(u,L,R);
        return max(query(2*u,L,M,l,r),query(2*u+1,M+1,R,l,r));
    }else{
        return -INF;
    }
}
```

2.6 ST 表

静态区间最大值 | 预处理 O(n log n)，查询 O(1) | 按个人风格补充
1-index，1<=n<N；改 N 时同步改 LOG；不能套用于区间和

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// 适用：数组不修改，反复询问一段区间的最大值。
// 调用例（放在 solve 中）：
// a[1]=5; a[2]=1; a[3]=7; a[4]=3; a[5]=2;
// build(5); cout << query(2,4); // 输出 7，查询含左右端点。
// 顺序：写入 a[1..n] -> build(n) 一次 -> 任意次 query(l,r)。
// 多测重新写 a 并 build 即可；修改 a 后原来的表不会自动更新。
// 要求 1<=l<=r<=n<N；LOG 层需覆盖 floor(log2(n))+1。
const int N = 200005,LOG = 18;
// f[j][i]：从 i 开始、长度为 2^j 的区间最大值。
ll a[N],f[LOG][N];
// lg[len]：不超过 len 的最大 2 的幂次，等于 floor(log2(len))。
int lg[N];
void build(int n){
    lg[1] = 0;
    for(int i=2 ; i<=n ; i++) lg[i] = lg[i/2]+1;
    for(int i=1 ; i<=n ; i++) f[0][i] = a[i];
    for(int j=1 ; j<=lg[n] ; j++){
        for(int i=1 ; i+(1<<j)-1<=n ; i++){
            // 长度 2^j 拆为前后两段，各长 2^(j-1)。
            f[j][i] = max(f[j-1][i],f[j-1][i+(1<<(j-1))]);
        }
    }
}
ll query(int l, int r){
    int k = lg[r-l+1];
    // 左右各取长度 2^k 的段，覆盖整段；重叠不影响 max。
    // 求和不能这样做：重叠部分会被重复计算。
    return max(f[k][l],f[k][r-(1<<k)+1]);
}
```

2.7 单调栈

右侧第一个严格更大元素的位置 | O(n) | 历史提取/修正
a[1..n]，n<N；不存在返回 0；大于等于时改为 >=

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 3e6+9;
ll a[N],stk[N],ans[N];
void build(int n){
    int top=0;
    fill(ans,ans+n+1,0);
    for(int i=1 ; i<=n ; i++){
        while(top>0 && a[i]>a[stk[top]]){
            ans[stk[top]]=i;
            top--;
        }
        stk[++top]=i;
    }
}
```

2.8 单调队列

长度 k 的滑动窗口最小值 | O(n) | 历史提取/修正
a[1..n]，1<=k<=n<N；窗口最大值把 <= 改成 >=

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 1e5+9;
ll a[N],q[N];
// 保留原两遍循环，只把 cout 的输出收集到 ans，便于其他题调用。
vector<ll> window_min(int n, int k){
    vector<ll> ans;
    int tail=0,head=1;
    for(int i=1 ; i<=n ; i++){
        while(head<=tail && i-q[head]+1>k) head++;
        while(head<=tail && a[i]<=a[q[tail]]) tail--;
        q[++tail]=i;
        if(i>=k) ans.push_back(a[q[head]]);
    }
    return ans;
}
vector<ll> window_max(int n, int k){
    vector<ll> ans;
    int head=1,tail=0;
    for(int i=1 ; i<=n ; i++){
        while(head<=tail && i-q[head]+1>k) head++;
        while(head<=tail && a[i]>=a[q[tail]]) tail--;
        q[++tail]=i;
        if(i>=k) ans.push_back(a[q[head]]);
    }
    return ans;
}
```

3.1 BFS

无权图最短距离 | O(n+m) | 按个人风格补充
1-index，n<N；不可达为 -1；多测清空 g[1..n]

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// 接口：bfs(n,st) 后直接读 d[v]，函数本身没有返回值。
// d[v] 是 st 到 v 最少经过的边数；st 为 0，不可达为 -1。
// 调用例（放在 solve 中）：
// g[1]={2}; g[2]={1,3}; g[3]={2};
// bfs(3,1); cout << d[3]; // 输出 2。
// 每组先清空 g[1..n] 再读边；bfs 会重置 d，可更换起点重复调用。
// 有向边只加 g[u].push_back(v)，无向边还要加反方向。
// 要求 1<=st<=n<N，所有邻点也在 1..n；只适用无权或等权图。
const int N = 200005;
vector<int> g[N];
int d[N];
void bfs(int n, int st){
    fill(d,d+n+1,-1);
    queue<int> q;
    q.push(st); d[st] = 0;
    while(!q.empty()){
        // 队列按距离从小到大处理，第一次发现 v 就得到最短距离。
        int u = q.front(); q.pop();
        for(int v : g[u]){
            if(d[v]!=-1) continue;
            // 入队时就标记，防止同一点被多个邻点重复加入。
            d[v] = d[u]+1;
            q.push(v);
        }
    }
}
```

3.2 迭代 DFS

遍历、生成树父亲与逆序树 DP | O(n+m) | 按个人风格补充
1-index，n<N；仅遍历 st 所在可达部分；多测清空 g

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// 用栈代替递归遍历；点编号为 1..n，1<=st<=n<N。
// dfs(n,st) 无返回值，结果在 order 和 fa 中。
// order 保存处理顺序，保证搜索树上的父亲排在孩子前面。
// fa[v] 为搜索树父亲；根的父亲为 0，不可达点为 -1。
// 调用例（放在 solve 中）：
// g[1]={2,3}; g[2]={1}; g[3]={1}; dfs(3,1);
// cout << fa[2]; // 输出 1；order 为 {1,3,2}。
// 树 DP 可倒序遍历 order，让所有孩子先算好，再更新父亲。
// 每组先清空 g[1..n] 再读边；dfs 会清空 order、重置 fa。
// 无向边存两次；只处理 st 可达部分，一般图得到一棵搜索树。
// 一般图的 order 不保证与逐条边递归 DFS 的顺序相同。
const int N = 200005;
vector<int> g[N],order;
int fa[N];
void dfs(int n, int st){
    fill(fa,fa+n+1,-1); order.clear();
    vector<int> stk = {st}; fa[st] = 0;
    while(!stk.empty()){
        int u = stk.back(); stk.pop_back();
        order.push_back(u);
        for(int v : g[u]){
            if(fa[v]!=-1) continue;
            // 入栈就确定父亲，避免环造成重复入栈；后入栈者先处理。
            fa[v] = u; stk.push_back(v);
        }
    }
}
// 树 DP 按 order 逆序处理；一般图这里只给出一棵搜索树。
```

3.3 拓扑排序

有向图判环、DAG 处理顺序 | O(n+m) | 历史提取/修正
1-index，n<N；topo_sort() 无返回值，ans_sort.size()!=n 表示有环；调用消耗 ind，重跑需重建入度

```
// ans_sort.size()==n 表示无环；函数会消耗 ind，重复调用前要重新统计入度。
#include<iostream>
#include<algorithm>
#include<queue>
#include<vector>
using namespace std;
using ll = long long;
const int N = 5e3+9;
vector<int> g[N];
int ind[N];
vector<int> ans_sort;
int n,m;
void topo_sort(){
    ans_sort.clear(); // 补充初始化：重复使用时清空上次答案。
    queue<int> q;
    for(int i=1 ; i<=n ; i++){
        if(ind[i]==0) q.push(i);
    }
    while (!q.empty())
    {
        int u = q.front(); q.pop();
        ans_sort.push_back(u);
        for(auto & v : g[u]){
            if(--ind[v]==0) q.push(v);
        }
    }
}
// 多测先清空 g[1..n]、ind[1..n]，然后重新建图和统计入度。
```

3.4 Dijkstra

非负权单源最短路 | O((n+m) log(m+2)) | 历史提取/修正
1-index，n<N；边权>=0、所需距离<INF；不可达 INF；多测清空 g

```
// 保留原函数名 dijksta：设好 n，建边 g[u].push_back({v,w})，调用 dijksta(s)。
#include<iostream>
#include<algorithm>
#include<vector>
#include<cstring>
#include<bitset>
#include<queue>
using namespace std;
using ll = long long;
const ll INF = 0x3f3f3f3f3f3f3f3f;
const int N = 1e5+9;
ll d[N];
bitset<N> vis;
struct edge{
    int v; ll w;
    bool operator < (const edge& u) const
    {
        if(w==u.w) return v<u.v;
        else return w>u.w;
    }
};
vector<edge> g[N];
priority_queue<edge> pq;
int n,m,s;
void dijksta(int st){
    memset(d,0x3f,sizeof(d));
    vis.reset(); // 原稿只跑一次；这里补充清空，方便重复调用。
    while(pq.size()) pq.pop();
    d[st]=0;
    pq.push({st,d[st]});
    while(pq.size()){
        int cur=pq.top().v; pq.pop();
        if(vis[cur]) continue;
        vis[cur]=true;
        for(auto &[v,w] : g[cur]){
            // 先判断减法，避免 d[cur]+w 溢出；距离 >=INF 不在本模板范围内。
            if(w>=INF-d[cur]) continue;
            if(!vis[v] && d[v]>d[cur]+w){
                d[v]=d[cur]+w;
                pq.push({v,d[v]});
            }
        }
    }
}
```

3.5 Floyd

全源最短路 | O(n^3) | 历史提取/修正
1-index，n<N；无负环，有限路径绝对值<INF/2；不可达 INF

```
// 先设 n 并 init()，再加边，最后调用 floyd()。
#include<iostream>
#include<algorithm>
#include<vector>
#include<cstring>
#include<bitset>
#include<queue>
using namespace std;
using ll = long long;
// 修正原稿：INF 与 memset 的 0x3f 保持一致，便于准确判断不可达。
const ll INF = 0x3f3f3f3f3f3f3f3f;
const int N = 305;
ll d[N][N];
int n;
void init(){
    memset(d,0x3f,sizeof(d));
}
void floyd(){
    for(int i=1 ; i<=n ; i++) d[i][i]=0;
    for(int k=1 ; k<=n ; k++){
        for(int i=1 ; i<=n ; i++){
            for(int j=1 ; j<=n ; j++){
                // 补充可达判断：INF 加负边不能当作一条真实路径。
                if(d[i][k]!=INF && d[k][j]!=INF)
                    d[i][j]=min(d[i][j],d[i][k]+d[k][j]);
            }
        }
    }
}
// init 后加边：d[u][v]=min(d[u][v],w)；无向边加两次。
```

3.6 Kruskal

最小生成树 / 最小生成森林 | 排序 O(m log m)，并查集仅路径压缩 | 历史提取/修正
1<=n<N，0<=m<N；填写静态边数组 e[1..m]，调用 kruskal(n,m)；返回 {是否连通,森林权值}

```
// 调用 kruskal(n,m)，返回 {是否连通,总权值}；不连通时权值为最小生成森林权值。
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 3e5+5;
struct edge{
    int u,v,w;
    bool operator < (const edge &oth) const{
        return w<oth.w;
    }
}e[N];
int fa[N];
int root(int x)
{
    if(fa[x]==x) return x;
    else return fa[x] = root(fa[x]);
}
pair<bool,ll> kruskal(int n, int m)
{
    for(int i=1 ; i<=n ; i++) fa[i] = i;
    sort(e+1,e+m+1);
    ll tt = 0;
    int cnt = 0;
    for(int i=1 ; i<=m ; i++){
        int u = root(e[i].u);
        int v = root(e[i].v);
        if(u==v) continue;
        // 原题的零权跳过、礼物价格限制均属题目逻辑；普通 MST 直接计入边权。
        tt += e[i].w;
        fa[u] = v;
        cnt++;
    }
    return {cnt==n-1,tt};
}
// 多测覆盖 e[1..m] 即可。root 保留递归路径压缩；极深父链要留意栈空间。
```

3.7 二分图染色

无向图二分性判定 | O(n+m) | 按个人风格补充
1-index，n<N；无向边加两次；覆盖不连通图

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// 判断无向图能否分成两组，使每条边的两端分属不同组。
// 返回 true 时 color[v] 为 0 或 1；false 表示存在冲突（奇环）。
// false 时染色可能尚未完成，不要把所有 color 当作合法分组。
// 调用例（放在 solve 中）：
// g[1]={2}; g[2]={1,3}; g[3]={2};
// bool ok=bipartite(3); // true，color 为 0,1,0。
// 再加 g[1].push_back(3); g[3].push_back(1); 会得到 false。
// 每组先清空 g[1..n] 再读边；bipartite 会自动重置 color。
// 编号 1..n，n<N；无向边必须存两个方向，自环会判为 false。
const int N = 200005;
vector<int> g[N];
int color[N];
bool bipartite(int n){
    fill(color,color+n+1,-1);
    queue<int> q;
    for(int st=1 ; st<=n ; st++){
        // 每个未染色的连通块都单独开始，孤立点也会染色。
        if(color[st]!=-1) continue;
        color[st] = 0; q.push(st);
        while(!q.empty()){
            int u = q.front(); q.pop();
            for(int v : g[u]){
                // ^1 在 0、1 之间切换；相邻点已同色就不可能合法。
                if(color[v]==-1){ color[v] = color[u]^1; q.push(v); }
                else if(color[v]==color[u]) return false;
            }
        }
    }
    return true;
}
```

3.8 二分图最大匹配

增广路求最大匹配数 | O(n左*m) | 按个人风格补充
左右分别 1..n、1..m，均<N；只存左到右边；递归深度 O(n)

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// 从已知二分图中选最多的边，要求任意两条选中边不共用端点。
// 左侧编号 1..n，右侧另从 1..m 编号；两侧同号不是同一个点。
// 只存左到右边 g[u].push_back(v)，这里不要加入反向边。
// 调用例（放在 solve 中）：
// g[1]={1,2}; g[2]={1}; cout << matching(2,2); // 输出 2。
// 结束后 match[1]=2、match[2]=1；match[v]=0 表示右点 v 未匹配。
// 每组先清空左侧 g[1..n] 再读边，最后调用 matching(n,m)。
// matching 会清空 match、vis；新增边后也可重新调用，重新求解。
// 要求 n,m<N；边必须跨两侧。递归深度可能达到左侧点数。
const int N = 2005;
vector<int> g[N];
// vis[v] 记录右点 v 最近在哪一轮增广中被访问，tag 是本轮编号。
int match[N],vis[N];
// 尝试给左点 u 找到配偶；成功返回 true，并修改沿途的匹配。
bool dfs(int u, int tag){
    for(int v : g[u]){
        if(vis[v]==tag) continue;
        vis[v] = tag;
        // 右点空闲就直接占用；否则尝试让它的旧配偶换一个右点。
        // 只有后续换位成功，才把当前 v 改配给 u，不会丢失旧匹配。
        if(!match[v] || dfs(match[v],tag)){
            match[v] = u; return true;
        }
    }
    return false;
}
int matching(int n, int m){
    fill(match,match+m+1,0); fill(vis,vis+m+1,0);
    int ans = 0;
    // 每个左点各尝试一次；本轮成功就使匹配边数增加 1。
    for(int u=1 ; u<=n ; u++) ans += dfs(u,u);
    return ans;
}
```

3.9 倍增 LCA

树 / 森林最近公共祖先 | 预处理 O(n log n)，查询 O(log n) | 按个人风格补充
1-index，n<N；每棵树取编号最小点为根；不同树返回 0

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// LCA 是两个点共同祖先中最深的点；适用于无向树或森林。
// 本模板每棵树以编号最小的点为根，build 不接受自定义根参数。
// 调用例（放在 solve 中）：
// g[1]={2,3}; g[2]={1}; g[3]={1}; g[4].clear(); build(4);
// cout << lca(2,3); // 输出 1；lca(2,4) 返回 0，表示不在同一棵树。
// 顺序：清空 g[1..n] -> 无向边存两次 -> build(n) -> 多次 lca。
// build 会重置查询状态；改边后必须重新 build，不能直接继续查询。
// 点编号必须为 1..n 且 n<N；图必须无环，不是一般图的公共祖先。
// 若增大 N，也要增大 LOG，使 2^LOG>最大可能深度。
const int N = 200005,LOG = 18;
vector<int> g[N];
// fa[j][v] 是 v 向上走 2^j 步的祖先，无祖先时为 0。
// dep[v] 是深度（根为 0）；bel[v] 是所在树的根，用于判断连通。
int fa[LOG][N],dep[N],bel[N];
void build(int n){
    fill(dep,dep+n+1,-1);
    for(int j=0 ; j<LOG ; j++) fill(fa[j],fa[j]+n+1,0);
    for(int st=1 ; st<=n ; st++){
        if(dep[st]!=-1) continue;
        queue<int> q; q.push(st); dep[st] = 0; bel[st] = st;
        while(!q.empty()){
            int u = q.front(); q.pop();
            for(int v : g[u]){
                if(dep[v]!=-1) continue;
                dep[v] = dep[u]+1; bel[v] = st; fa[0][v] = u;
                // 走 2^j 步 = 先走 2^(j-1) 步，再走相同步数。
                for(int j=1 ; j<LOG ; j++) fa[j][v] = fa[j-1][fa[j-1][v]];
                q.push(v);
            }
        }
    }
}
int lca(int u, int v){
    if(bel[u]!=bel[v]) return 0;
    if(dep[u]<dep[v]) swap(u,v);
    int d = dep[u]-dep[v];
    // 先按深度差的二进制位跳，让 u、v 到达同一层。
    for(int j=0 ; j<LOG ; j++) if(d>>j&1) u = fa[j][u];
    if(u==v) return u;
    for(int j=LOG-1 ; j>=0 ; j--){
        // 只跳到仍不同的祖先；最后两点的父亲就是 LCA。
        if(fa[j][u]!=fa[j][v]) u = fa[j][u],v = fa[j][v];
    }
    return fa[0][u];
}
```

3.10 强连通分量

Kosaraju 迭代版 | O(n+m) | 按个人风格补充
1-index，n<N；加边同时 g[u].push_back(v)、rg[v].push_back(u)

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// 有向图中，能互相到达的一组点构成一个强连通分量。
// scc(n) 无返回值；cnt 是分量数，bel[v] 是点 v 所属编号（1..cnt）。
// bel[u]==bel[v] 表示互相可达；孤立点也单独算一个分量。
// 调用例（放在 solve 中）：
// for(int i=1;i<=3;i++){ g[i].clear(); rg[i].clear(); }
// auto add=[&](int u,int v){ g[u].push_back(v); rg[v].push_back(u); };
// add(1,2); add(2,1); add(2,3); scc(3);
// 此时 cnt=2，bel[1]==bel[2]，bel[2]!=bel[3]。
// 每组先清空 g、rg 的 1..n，再同时建立原图和反图，最后 scc(n)。
// scc 会重置所有工作数组；要求点编号为 1..n，n<N。
// 缩点：原边 u->v 且 bel[u]!=bel[v] 时，连 bel[u]->bel[v]。
// 缩点图一定是 DAG；重复边是否去重由后续题意决定。
const int N = 200005;
// g 为原图，rg 为所有边反向的图；order 最初保存 DFS 完成顺序。
vector<int> g[N],rg[N],order;
int vis[N],bel[N],cnt;
void scc(int n){
    fill(vis,vis+n+1,0); fill(bel,bel+n+1,0);
    order.clear(); cnt = 0;
    for(int st=1 ; st<=n ; st++){
        if(vis[st]) continue;
        // 栈中 {u,i} 模拟递归：u 下一次要检查的是第 i 条出边。
        vector<pair<int,int>> stk = {{st,0}}; vis[st] = 1;
        while(!stk.empty()){
            int u = stk.back().first;
            int &i = stk.back().second;
            if(i==(int)g[u].size()){
                // 必须等所有出边处理完再记 u，这是“退出”顺序。
                order.push_back(u); stk.pop_back(); continue;
            }
            int v = g[u][i++];
            if(!vis[v]){ vis[v] = 1; stk.push_back({v,0}); }
        }
    }
    // 按退出顺序倒序，在反图遍历；每次新起点恰好找到一个分量。
    // 反图中遇到已分配 bel 的点就不再进入，避免跨入已处理分量。
    reverse(order.begin(),order.end());
    for(int st : order){
        if(bel[st]) continue;
        vector<int> stk = {st}; bel[st] = ++cnt;
        while(!stk.empty()){
            int u = stk.back(); stk.pop_back();
            for(int v : rg[u]){
                if(!bel[v]){ bel[v] = cnt; stk.push_back(v); }
            }
        }
    }
}
```

3.11 割点与桥

无向图 lowlink，支持重边 | O(n+m) | 按个人风格补充
1-index，n<N；每条无向边有唯一非负 id，两方向共用 id

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// 割点：删去该点及其边后连通块数增加；桥：删去该边后块数增加。
// 只用于无向图；支持重边，点编号为 1..n 且 n<N。
// 每条无向边给唯一非负 id，同一条边的两个方向共用这个 id。
// 调用例（放在 solve 中）：
// for(int i=1;i<=3;i++) g[i].clear();
// g[1].push_back({2,0}); g[2].push_back({1,0});
// g[2].push_back({3,1}); g[3].push_back({2,1}); build(3);
// 此时 cut[2]=true，bridge 包含边号 0、1（顺序不保证）。
// build 无返回值；cut[v] 表示是否割点，bridge 保存所有桥的 id。
// 每组清空 g[1..n]、重新加边后调用 build；内部会重置工作数组。
// 若要输出桥的两个端点，读边时另外按 id 保存端点，凭 id 查回。
const int N = 200005;
vector<pair<int,int>> g[N]; // {邻点,边编号}
// dfn[u] 为首次访问时间；low[u] 为 u 的子树经树边和至多一条
// 非树边能到达的最小 dfn。low 越小，说明能绕回越早的祖先。
// fa[u] 为 DFS 父亲，pe[u] 为父边 id；根 fa=0、pe=-1。
// it[u] 为下一条待检查边；child[u] 为 DFS 树中直接孩子数。
int dfn[N],low[N],fa[N],pe[N],it[N],child[N],timer;
bool cut[N];
vector<int> bridge;
void build(int n){
    timer = 0; bridge.clear();
    for(int i=1 ; i<=n ; i++){
        dfn[i] = low[i] = fa[i] = it[i] = child[i] = 0;
        pe[i] = -1; cut[i] = false;
    }
    for(int st=1 ; st<=n ; st++){
        if(dfn[st]) continue;
        vector<int> stk = {st}; dfn[st] = low[st] = ++timer;
        while(!stk.empty()){
            int u = stk.back();
            if(it[u]<(int)g[u].size()){
                auto [v,id] = g[u][it[u]++];
                // 只跳过父边本身，不能跳过所有通往父亲的边（重边）。
                if(id==pe[u]) continue;
                if(!dfn[v]){
                    fa[v] = u; pe[v] = id; child[u]++;
                    dfn[v] = low[v] = ++timer; stk.push_back(v);
                }else low[u] = min(low[u],dfn[v]);
            }else{
                // 出栈相当于递归返回，u 的所有孩子已完成，low[u] 已确定。
                stk.pop_back(); int p = fa[u];
                // 根没有祖先；有至少两个 DFS 孩子时，删根才会分裂。
                if(!p){ cut[u] = child[u]>1; continue; }
                low[p] = min(low[p],low[u]);
                // 子树连父亲也绕不到，父边就是桥（严格大于）。
                if(low[u]>dfn[p]) bridge.push_back(pe[u]);
                // 非根父亲：子树绕不到父亲以上，删父亲会断开（可等于）。
                if(fa[p] && low[u]>=dfn[p]) cut[p] = true;
            }
        }
    }
}
```

3.12 A* 第 k 短路

非负权图第 k 短游走，可重复点边 | 最坏 O((n+km) log(n+km)) | 历史提取/修正
g2 存反图；1<=n<N，1<=原 k<INT_MAX；s==t 时先 k++ 排除空路径；先 dijkstra(t) 再 Astar(s)；边权>=0、答案<INF

```
// 建边：g1[a].push_back({b,l}); g2[b].push_back({a,l});
// 设好 s,t,k；若 s==t，先 k++ 排除空游走，然后 dijkstra(t); Astar(s);
// 答案在 ans 中，-1 表示本范围内不存在；多测清空 g1/g2。
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 1010;
const ll INF = 0x3f3f3f3f3f3f3f3f;
bitset<N> vis;
int n,m;
int s,t,k;
struct edge{
    int v;
    ll l;
    bool operator < (const edge& u) const
    {
        return l>u.l;
    }
};
ll dist[N];
vector<edge> g1[N];
vector<edge> g2[N];
ll ans = -1;
int cnt = 0;
int times[N]; // 补充：各点已弹出次数，防止含环时反复扩展而无法结束。
void dijkstra(int st)
{
    vis.reset(); // 补充初始化，允许重复调用。
    for(int i=1 ; i<=n ; i++) dist[i] = INF;
    dist[st] = 0;
    priority_queue<edge> pq;
    pq.push({st,dist[st]});
    while(!pq.empty()){
        int cur = pq.top().v; pq.pop();
        if(vis[cur]) continue;
        vis[cur] = true;
        for(auto [v,l] : g2[cur]){
            // 先用减法判断范围，保证后面的 ll 加法安全。
            if(l>=INF-dist[cur]) continue;
            if(dist[cur] + l < dist[v]){
                dist[v] = dist[cur] + l;
                pq.push({v,dist[v]});
            }
        }
    }
}
void Astar(int st)
{
    ans = -1;
    cnt = 0;
    memset(times,0,sizeof(times));
    if(dist[st]==INF) return; // 补充：起点无法到终点时不用搜索。
    priority_queue<edge> pq;
    pq.push({st,dist[st]});
    while(!pq.empty()){
        int cur = pq.top().v;
        ll curl = pq.top().l;
        ll f = curl - dist[cur];
        pq.pop();
        // 第 k 短只需要每个点前 k 次弹出，零权环也能终止。
        if(times[cur]>=k) continue;
        times[cur]++;
        if(cur==t) cnt++;
        if(cnt==k){
            ans = curl;
```

3.12 A* 第 k 短路 (续)

02_graph/astar_kth_walk.cpp

```
            return;
        }
        for(auto [v,l] : g1[cur]){
            if(dist[v]==INF || times[v]>=k) continue;
            // curl 是估计总长，f 是已走长度；保持原来的 f+dist[v]+l。
            // 边权非负，分两步保证总和 <INF，不需要更宽的整数类型。
            if(l>=INF-f) continue;
            if(dist[v]>=INF-f-l) continue;
            pq.push({v,f+dist[v]+l});
        }
    }
}
```

4.1 KMP

拼接串前缀函数，输出匹配起点和模式串 pi | O(n+m) | 历史提取/修正
原程序：读入正文 s1、模式 s2；两串非空且不含 #；输出起点为 1-index

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    string s1,s2; cin>>s1>>s2;
    string s = s2 + '#' + s1;
    vector<int> pi(s.length());
    for(int i=1 ; i<s.length() ; i++){
        int len = pi[i-1];
        while(s[i]!=s[len] && len){
            len = pi[len-1];
        }
        if(s[i] == s[len]){
            pi[i] = len+1;
        }
        if(pi[i] == s2.length()) cout<<i-2*s2.length()+1<<'\n';
    }
    for(int i=0 ; i<s2.length() ; i++) cout<<pi[i]<<" ";
    return 0;
}
```

4.2 Manacher

最长回文子串长度 | O(n) | 历史提取/修正
原程序：读入非空字符串；输入不可含 @/#/$；输出最长回文长度

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    string ss; cin>>ss;
    string s = "@";
    for(char c : ss){
        s += '#';
        s += c;
    }
    s += "#$";
    int n = s.length();
    vector<int> p(n);
    int C=0,R=0;
    for(int i=1; i<n-1; i++){
        int j = 2*C-i;
        if(i<R) p[i] = min(R-i,p[j]);
        while(s[i+p[i]+1] == s[i-p[i]-1]) p[i]++;
        if(i+p[i] > R){
            C = i;
            R = i+p[i];
        }
    }
    int ans =  *max_element(p.begin(),p.end());
    cout<<ans;
    return 0;
}
```

4.3 Trie

大小写字母字典，统计完整单词次数 | O(字符串长度) | 历史提取/修正
只允许 A-Z/a-z；最多 N-1 个非根节点，约 212MB；空间不足时减 N

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 1000005;
int trie[N][52],cnt[N];
int n, tot = 0;
void insert(string s){
    int u = 0;
    int len = s.length();
    for(int i=0; i<len; i++){
        if('a'<=s[i] && s[i]<='z'){
            int a = s[i] - 'a';
            if(trie[u][a] == 0)
                trie[u][a] = ++tot;
            u = trie[u][a];
        }else{
            int a = s[i] - 'A' + 26;
            if(trie[u][a] == 0)
                trie[u][a] = ++tot;
            u = trie[u][a];
        }
    }
    cnt[u]++;
}
// 以下补充查询和多测清空，insert 保留原来的大小写分支。
int query(string s){
    int u = 0;
    int len = s.length();
    for(int i=0; i<len; i++){
        int a;
        if('a'<=s[i] && s[i]<='z') a = s[i]-'a';
        else a = s[i]-'A'+26;
        if(trie[u][a] == 0) return 0;
        u = trie[u][a];
    }
    return cnt[u];
}
void init(){
    for(int i=0; i<=tot; i++){
        fill(trie[i],trie[i]+52,0);
        cnt[i] = 0;
    }
    tot = 0;
}
```

4.4 Z-function

各后缀与全串的最长公共前缀 | O(n) | 按个人风格补充
0-index；本模板约定 z[0]=n

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// z[i]：从 s[i] 开始的后缀与整个 s 的最长公共前缀长度。
// 调用例（放在 solve 中）：
// auto z=z_function("abcab"); // 得到 {5,0,0,2,0}。
// z[3]=2，因为 s[3..4]="ab" 与 s 的前两个字符相同。
// 下标从 0 开始；本模板 z[0]=s.size()，空串返回空 vector。
// 也可做匹配：对 p+'#'+t 求 z，文本位置对应的 z>=p.size() 即匹配。
// 例如 z_function("ab#ababa") 的 3、5 位置为 2，对应文本起点 0、2。
// 用拼接法时 '#' 必须不在 p、t 中；只求 z 本身没有此限制。
// 结果直接返回；所有状态均在函数内，多测不需要额外清空。
vector<int> z_function(const string& s){
    int n = s.size(); vector<int> z(n);
    for(int i=1,L=0,R=0 ; i<n ; i++){
        // [L,R] 是已知与前缀相同且右端最远的一段（闭区间）。
        // i 在段内时先复用 z[i-L]，但最多只能保证到 R。
        if(i<=R) z[i] = min(R-i+1,z[i-L]);
        // 从已确定的长度继续匹配；只有越过旧 R 的比较才会不断增长。
        while(i+z[i]<n && s[z[i]]==s[i+z[i]]) z[i]++;
        if(i+z[i]-1>R){ L = i; R = i+z[i]-1; }
    }
    if(n) z[0] = n;
    return z;
}
```

4.5 双模字符串哈希

子串相等性快速筛选 | 预处理 O(n)，查询 O(1) | 按个人风格补充
0-index 半开 [l,r)；比较时同时比较长度；仍可能碰撞，非确定性证明

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// 为静态字符串预处理“指纹”，随后 O(1) 取任意子串的双模哈希。
// 调用例（放在 solve 中）：
// Hash h; h.init("ababa");
// bool same=(h.query(0,3)==h.query(2,5)); // true，两段都是 "aba"。
// query(l,r) 使用半开区间 [l,r)，包含 l、不包含 r，长度为 r-l。
// 要求先 init 且 0<=l<=r<=字符串长度；空区间返回 {0,0}。
// 比较子串时必须先保证长度相等，再比较返回的 pair<ll,ll>。
// 不同字符串可各建一个 Hash 对象；相同参数下可直接比较等长子串。
// 哈希相同仍可能碰撞；需要绝对正确时再比较原子串，会增加耗时。
// 多测再次 init 即可；修改原串后必须重新 init，不能直接沿用。
const ll MOD1 = 1000000007,MOD2 = 1000000009,B = 131;
struct Hash{
    // h1[i]/h2[i] 为前 i 个字符的哈希；p1[i]/p2[i] 为 B^i 取模。
    vector<ll> h1,h2,p1,p2;
    void init(const string& s){
        int n = s.size();
        h1.assign(n+1,0); h2.assign(n+1,0);
        p1.assign(n+1,1); p2.assign(n+1,1);
        for(int i=1 ; i<=n ; i++){
            // 按字节编码，并加 1 使字符值非零；前缀向左移一位再加字符。
            int c = (unsigned char)s[i-1]+1;
            h1[i] = (h1[i-1]*B+c)%MOD1;
            h2[i] = (h2[i-1]*B+c)%MOD2;
            p1[i] = p1[i-1]*B%MOD1; p2[i] = p2[i-1]*B%MOD2;
        }
    }
    pair<ll,ll> query(int l, int r){
        // 从前 r 位减去前 l 位的贡献；乘 B^(r-l) 对齐后半段位数。
        // 加 MOD 再取模，把减法结果调整到非负；乘积可装入 ll。
        return {(h1[r]-h1[l]*p1[r-l]%MOD1+MOD1)%MOD1,
                (h2[r]-h2[l]*p2[r-l]%MOD2+MOD2)%MOD2};
    }
};
```

5.1 埃氏筛

筛出 2..n 所有素数 | O(n log log n) | 历史提取/修正
0<=n<N；vis=1 表示非素数；保留从 2*i 开始，只修正原 vis[i] 笔误

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 2e6+9;
bitset<N> vis;
vector<int> pri;
void sieve(int n){
    vis.reset(); pri.clear();
    vis[0]=1,vis[1]=1;
    for(int i=2 ; i<=n ; i++){
        if(!vis[i]){
            for(int j=2*i ; j<=n ; j+=i) vis[j]=1;
        }
    }
    for(int i=1 ; i<=n ; i++) if(!vis[i]) pri.push_back(i);
}
```

5.2 试除分解质因数

返回 {素因子,指数} | O(sqrt(n)) | 历史提取/修正
n>=1；大整数试除可能超时，适合约 1e12 以内的少量查询

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
vector<pair<ll,int>> factor(ll n){
    vector<pair<ll,int>> ans;
    for(ll i=2 ; n/i>=i ; i++){
        if(n%i==0){
            int cnt=0;
            while(n%i==0){
                cnt++;
                n/=i;
            }
            ans.push_back({i,cnt});
        }
    }
    if(n>1) ans.push_back({n,1});
    return ans;
}
```

5.3 扩展欧几里得与逆元

ax+by=gcd(a,b)，一般模数逆元 | O(log max(a,b)) | 按个人风格补充
exgcd: 0<=a,b<=1e9 且不全为0；inv: 2<=p<=1e9；无逆元返回 -1

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// 调用示例：ll x,y; ll d=exgcd(30,18,x,y);
// 得到 30*x+18*y=d=6；解不唯一，这里返回其中一组。
// x,y 用引用传出；函数返回最大公约数，不是逆元。
ll exgcd(ll a, ll b, ll& x, ll& y){
    if(b==0){
        x = 1; y = 0;
        return a;
    }
    ll d = exgcd(b,a%b,y,x);
    // 递归已求出 b*y+(a%b)*x=d。
    // 用 a%b=a-(a/b)*b 代入，得到当前 a,b 的系数。
    y -= (a/b)*x;
    return d;
}
// inv(3,11)=4，因为 3*4%11=1；inv(6,9)=-1，因为不互质。
// 模数不必是素数；只有 gcd(a,p)=1 才有逆元。
ll inv(ll a, ll p){
    if(p<=1) return -1;
    a %= p;
    if(a<0) a += p; // 先把负数化到 [0,p)
    ll x,y;
    if(exgcd(a,p,x,y)!=1) return -1;
    x %= p;
    if(x<0) x += p;
    return x;
}
// 普通 gcd 用 std::gcd(a,b)；本文件没有全局状态，不需要清空。
```

5.4 组合数

阶乘与快速幂逆元求组合数 | 预处理 O(n)，查询 O(log MOD) | 历史提取/修正
先 init(最大 n)，0<=n<N 且 n<MOD；保留原 inv 快速幂和单次求逆

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 1e7+9;
const int MOD = 1e9+7;
ll f[N];
ll qpow(ll a , ll n){
    ll res=1;
    while(n>0){
        if(n%2==1) res=res*a%MOD;
        a=a*a%MOD;
        n/=2;
    }
    return res;
}
ll inv(ll x){
    return qpow(x,MOD-2);
}
void init(int n){
    f[0]=1;
    for(int i=1 ; i<=n ; i++) f[i]=i*f[i-1]%MOD;
}
ll c(ll n , ll m){
    if(m<0 || m>n) return 0;
    return f[n]*inv(f[m])%MOD*inv(f[n-m])%MOD;
}
```

6.1 01 背包

容量至多 m 的最大价值 | O(nm) | 历史提取/修正
物品 {体积,价值}，体积>=0；恰好装满需负 INF 初始化并判可达

```
#include<iostream>
#include<algorithm>
using namespace std;
using ll = long long;
const int N = 1e3+9;
ll dp[N];
// 把原来读到一件物品后的更新单独拿出来，仍用原 dp 和倒序循环。
void add_item(int T, int t, int v){
    for(int j=T ; j>=t ; j--){
        dp[j]=max(dp[j],dp[j-t]+v);
    }
}
// 原用法：
// int T,n; cin>>T>>n;
// fill(dp,dp+T+1,0); // 多测必须清空
// for(int i=1 ; i<=n ; i++){
//     int t,v; cin>>t>>v;
//     add_item(T,t,v);
// }
// cout<<dp[T];
// 恰好装满时 dp[0]=0，其余置负 INF；转移前必须判断 dp[j-t] 可达。
```

6.2 完全背包计数

每种体积无限次，恰好凑出 n 的组合数 | O(种类数*n) | 历史提取/修正
0<=n<N；pri 放互不相同的正体积；knapsack() 初始化 dp[0]=1，答案为 dp[n]；所有计数须装入 ll

```
#include<iostream>
#include<algorithm>
#include<vector>
using namespace std;
using ll = long long;
vector<int> pri;
const int N = 1e3+9;
ll dp[N];
int n;
// pri 沿用原变量名：原题存素数，这里可以填任意允许使用的正体积。
// 原题的素数判断、读入和输出去掉，背包循环照原样保留。
void knapsack(){
    fill(dp,dp+n+1,0);
    dp[0]=1;
    for(int i=0 ; i<pri.size() ; i++){
        for(int j=pri[i] ; j<=n ; j++){
            dp[j]+=dp[j-pri[i]];
        }
    }
}
// 用法：n=5; pri={2,3}; knapsack(); cout<<dp[n]; // 1 种：2+3
// dp[j] 表示已枚举体积组成 j 的方案数；外层体积、内层正序。
// 正序使本种体积可以反复用；外层枚举种类避免把 2+3、3+2 重复计数。
// 若题目要求取模，在这条加法转移中按题意取模。
```

6.3 多重背包

二进制拆分数量限制 | O(m * sum(log(cnt+1))) | 按个人风格补充
t>0，cnt>=0，m>=0；价值乘数量及总价值必须装入 ll

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// 每种物品最多 cnt 件，求总容量不超过 m 时能取得的最大价值。
// item 中 t=单件体积，v=单件价值，cnt=最多可用件数。
// 调用例（放在 solve 中）：
// vector<item> a={{2,3,2},{3,5,1}};
// cout << knapsack(7,a); // 输出 11：第一种 2 件，第二种 1 件。
// 返回最大价值 ll，不返回选择方案；允许一件不选，所以结果至少为 0。
// 要求 m>=0、t>0、cnt>=0，容量数组可放入内存，价值运算不溢出 ll。
// 函数内部新建 dp，多测直接重新调用；输入物品数量不会被修改。
// 此处是“至多装 m”；恰好装满时要另改不可达初始化和转移判断。
struct item{ int t; ll v; int cnt; };
ll knapsack(int m, const vector<item>& a){
    // dp[j]：已处理物品、容量至多 j 的最大价值，初始都为 0。
    vector<ll> dp(m+1);
    for(auto [t,v,cnt] : a){
        // 再多也装不下的件数先去掉，且保证后面的 t*c 不超过 m。
        cnt = min(cnt,m/t);
        // 拆成 1、2、4 等大小的包和最后余数，每包当作一个 01 物品。
        // 例如 13 件拆为 1、2、4、6，可组合出 0..13 的任意件数。
        for(ll k=1 ; cnt>0 ; k*=2){
            int c = (int)min(k,(ll)cnt); cnt -= c;
            int w = t*c; ll val = v*c;
            // 容量倒序，避免同一个拆分包在本轮被重复使用。
            for(int j=m ; j>=w ; j--) dp[j] = max(dp[j],dp[j-w]+val);
        }
    }
    return dp[m];
}
```

6.4 分组背包

每组至多选一个物品 | O(m * 总物品数) | 历史提取/修正
0<=m<N，组号 1..z_max<105；a 为正体积，b 为价值；原地倒序转移不支持零体积；多测清空 wp

```
#include<iostream>
#include<algorithm>
#include<vector>
using namespace std;
using ll = long long;
const int N = 1e3+9;
struct w{
    int a,b;
};
ll dp[N];
vector<w> wp[105];
ll knapsack(int m, int z_max){
    fill(dp,dp+m+1,0);
    for(int i=1 ; i<=z_max ; i++){
        for(int j=m ; j>=0 ; j--){
            for(auto &c : wp[i]){
                if(j>=c.a){
                    dp[j]=max(dp[j],dp[j-c.a]+c.b);
                }
            }
        }
    }
    return dp[m];
}
// 沿用原用法：wp[z].push_back({x,y})，x 是体积，y 是价值。
// 先算出 z_max，再调用 knapsack(m,z_max)；多测先清空各组 wp。
// 保留原来的容量倒序写法，必须 a>0，保证 dp[j-a] 仍是上一组状态。
```

6.5 最长严格上升子序列

长度与一个下标方案 | O(n log n) | 按个人风格补充
返回 0-index 下标；非降序列把 lower_bound 改为 upper_bound

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// 求最长严格上升子序列：可以跳过元素，但不能改变原来的先后顺序。
// 调用例（放在 solve 中）：
// vector<ll> a={3,1,2,2,4}; auto ans=lis(a);
// cout << ans.size(); // 输出长度 3。
// for(int i:ans) cout << a[i] << ' '; // 输出一组方案 1 2 4。
// 返回的是原数组下标（从 0 开始），不是数值；有多解时只返回其中一组。
// 空数组返回空 vector；状态全部在函数内，多测直接调用。
// 如果允许相等，即求非降子序列，将 lower_bound 改为 upper_bound。
vector<int> lis(const vector<ll>& a){
    // d[j] 是长度 j+1 的上升子序列的最小末尾值；d 本身不一定是方案。
    vector<ll> d;
    // pos[j] 是 d[j] 在原数组的位置；pre[i] 记录方案中 i 的前驱下标。
    vector<int> pos,pre(a.size(),-1),ans;
    for(int i=0 ; i<(int)a.size() ; i++){
        // 找首个 >=a[i] 的末尾替换，使相同长度更容易在后面接新元素。
        int j = lower_bound(d.begin(),d.end(),a[i])-d.begin();
        // 接在长度 j 的方案后；必须先记前驱，再修改本层 pos。
        if(j) pre[i] = pos[j-1];
        if(j==(int)d.size()){ d.push_back(a[i]); pos.push_back(i); }
        else{ d[j] = a[i]; pos[j] = i; }
    }
    if(pos.empty()) return ans;
    // 从最长方案的末尾沿 pre 倒着追溯，最后反转恢复原下标顺序。
    for(int u=pos.back() ; u!=-1 ; u=pre[u]) ans.push_back(u);
    reverse(ans.begin(),ans.end());
    return ans;
}
```

6.6 子集枚举

遍历一个 mask 的所有子集 | O(2^popcount(mask)) | 按个人风格补充
含空集；所有 mask 再枚举子集总计 O(3^n)，不要保存全部结果

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// 用二进制的第 i 位表示是否选第 i 个元素，mask 表示允许选择的集合。
// 只枚举 mask 中已有元素组成的子集，包含 mask 本身和空集 0。
// 调用例（放在 solve 中）：
// vector<unsigned> a;
// submasks(0b101u,[&](unsigned s){ a.push_back(s); });
// a 依次为 {5,4,1,0}，即 {0,2}、{2}、{0}、空集。
// [&] 让括号内代码可读写外部变量；把 push_back 换成自己的计算即可。
// 函数没有返回值，每得到一个子集就调用一次 work(s)，顺序从大到小。
// mask=0 时也会调用一次 work(0)；内部无全局状态，多测直接调用。
// unsigned 通常为 32 位；用 1u<<i 表示元素 i 时要求 0<=i<32。
// 若对 n 个元素的所有 mask 再枚举子集，总量 O(3^n)，只适合小 n。
template<class F>
void submasks(unsigned mask, F work){
    // s-1 借位产生下一个候选，再 &mask 去掉不在原集合中的位。
    for(unsigned s=mask ; ; s=(s-1)&mask){
        work(s);
        // 处理 0 后立即结束；否则无符号减法绕回，重新得到 mask。
        if(s==0) break;
    }
}
// n<32 时用 1u<<n；__builtin_ctz(x) 要求 x!=0。
```

7.1 二维凸包

返回上凸壳 hi 与下凸壳 lo，两条链均按 x 递增 | O(n log n) | 历史提取/修正
整数坐标 |x|,|y|<=1e9，叉积和距离平方使用 ll；同 x 上壳留最高点、下壳留最低点；共线中间点被弹出；支持空集/单点

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
struct Point{
    ll x;
    ll y;
    Point() : x{0},y{0} {}
    Point(ll x_, ll y_) : x{x_},y{y_}{}
};
ll cross(Point a, Point b){
    return a.x*b.y - a.y*b.x;
}
Point operator+(Point a, Point b){
    return Point(a.x+b.x,a.y+b.y);
}
Point operator-(Point a, Point b){
    return Point(a.x-b.x,a.y-b.y);
}
// 沿用原 dis：返回距离平方，不是欧氏距离。
ll dis(Point a, Point b){
    return (a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-b.y);
}
auto getHull(vector<Point> p){
    sort(p.begin(),p.end(),
    [&](auto a, auto b){
        return a.x<b.x || (a.x==b.x && a.y<b.y);
    });
    vector<Point> hi,lo;
    for(auto p : p){
        while(hi.size()>1 &&
              cross(hi.back()-hi[hi.size()-2] , p-hi.back() ) >= 0) hi.pop_back();
        while(!hi.empty() && hi.back().x == p.x) hi.pop_back();
        hi.push_back(p);
        while(lo.size()>1 &&
              cross(lo.back()-lo[lo.size()-2] , p-lo.back() ) <= 0) lo.pop_back();
        if(lo.empty() || lo.back().x < p.x) lo.push_back(p);
    }
    return make_pair(hi,lo);
}
// auto [hi,lo] = getHull(a); 两条链都按 x 递增，hi 是上壳、lo 是下壳。
// 它们不是一个按逆时针排列的点集；需要闭合边界时还要连两端。
// 同 x 的点：hi 留最高点、lo 留最低点；共线中间点被弹出。
// 空输入返回两条空链，调用 front()/back() 前先判空。
// 原 P2742 的实数坐标版把 Point/cross 改成 double，dis 再开根。
```